

Complete Technical Reference

Project	sap-ai-assistant
SAP System	ISD (SNC / Kerberos)
Packages	/HEC1/CP_CDD /HEC1/CP_NCDD
Java Server	Spring Boot 3.2 · Port 8080
Pipeline	Node.js 18+ (no external deps)
Auth	SNC-only (no username/password at runtime)
Generated	2026-04-17

1. Project Overview

SAP AI Assistant is a locally-hosted intelligence layer that sits on top of your SAP ABAP development landscape. It combines a **Java MCP (Model Context Protocol) server** with a **Node.js knowledge pipeline** and a **local offline code cache** to give AI assistants (Claude Code, etc.) rich, structured access to your ABAP code — even when SAP or the internet is unavailable.

- Core objectives:**
- Offline-first: all CDD/NCDD code and architecture knowledge available without any network connection.
 - SNC-only authentication: no username/password ever stored; Kerberos/GSS-API only.
 - Single one-time crawl: fetch all source once from SAP ISD, then work entirely from local cache.
 - 88 MCP tools covering read, write, testing, debugging, transport, and data operations.
 - Pipeline-generated knowledge base injected into AI context via `/cdd-knowledge slash` command.

Technology stack

Layer	Technology	Purpose
Java MCP Server	Spring Boot 3.2.1, Java 17, JCo RFC	SAP connectivity, SNC auth, 88 MCP tools
Knowledge Pipeline	Node.js 18+ (zero npm deps)	Crawl → analyse → export ABAP knowledge
Offline Cache	JSON files + index.json	Local code store served by <code>cdd_local_get_code</code>
AI Interface	Claude Code CLI + MCP protocol	Consumer of all tools; uses offline cache first
SAP System	ISD (SNC/Kerberos)	Source of truth for ABAP code (crawl only)

2. Repository Structure

The project lives at `~/Desktop/sap-ai-assistant/`. Key directories:

Path	Description
<code>jco-service/</code>	Java Spring Boot application (Maven). The MCP server binary.
<code>jco-service/src/main/java/.../handlers/</code>	88 MCP tool handlers (read, write, test, debug, data...)
<code>jco-service/src/main/java/.../service/</code>	Core services: <code>AdtClient</code> , <code>JcoSessionManager</code> , <code>CddCodeCacheService</code>
<code>jco-service/src/main/java/.../config/</code>	Config beans: <code>CddCacheConfig</code> , <code>SystemConfigLoader</code>
<code>jco-service/src/main/java/.../web/</code>	REST controllers: <code>ChatController</code> , <code>PipelineController</code>
<code>jco-service/src/main/resources/</code>	<code>application.properties</code> , <code>cdd-knowledge-base.md</code> (bundled)
<code>jco-service/target/jco-service-1.0.0.jar</code>	Compiled runnable JAR (after mvn package)
<code>pipeline/</code>	Node.js knowledge pipeline (no npm deps required)
<code>pipeline/cdd-crawler.js</code>	Main crawler: phases 1, 2, 3
<code>pipeline/graph-builder.js</code>	Builds <code>edges.json</code> , <code>mind-map.json</code> from crawled records
<code>pipeline/export-builder.js</code>	Produces <code>cdd-system-prompt.md</code> + <code>cdd-rag-chunks.jsonl</code>

Path	Description
pipeline/sync-java-cache.js	Syncs pipeline output to Java offline cache
pipeline/fix-manifest.js	Manifest repair after interrupted crawl
pipeline/pipeline-config.json	Crawler config: system, concurrency, paths
knowledge/	Generated at runtime (gitignored). Holds all crawled data.
knowledge/abap/{type}/	One JSON per ABAP object (class, interface, program, etc.)
knowledge/graph/	index.json, edges.json, mind-map.json, top-connected.json
knowledge/crawl/	manifest.json (progress state), crawl.lock (PID)
knowledge/exports/	cdd-system-prompt.md, cdd-rag-chunks.jsonl
~/sap-ai-assistant/cdd-cache/	Java offline cache (outside repo). Served by MCP tools.
.claude/commands/cdd-knowledge.md	Slash command: /cdd-knowledge injects system prompt
.sap-systems.json	SAP system registry (gitignored, SNC credentials)
docs/	Architecture, tools reference, migration, troubleshooting
start-sap-assistant.sh	Convenience launcher (sets DYLD_LIBRARY_PATH)

3. System Architecture

The system has two distinct runtime modes:

Crawl mode (one-time setup, SAP required)

The Node.js pipeline calls the Java server at `http://localhost:8080/api/pipeline/tool`. The Java server uses JCo SNC authentication to connect to SAP ISD and retrieve source code. Results are saved locally as JSON files. Claude, the internet, and hai proxy are NOT needed.

Offline mode (daily use, zero external dependencies)

The Java MCP server reads from `~/sap-ai-assistant/cdd-cache/`. All three CDD/NCDD tools (`cdd_knowledge_base`, `cdd_local_search`, `cdd_local_get_code`) answer without any network call. The `/cdd-knowledge` slash command injects the pre-generated system prompt directly into the AI context.

Data flow diagram

Stage	Component	Input	Output
Phase 1	cdd-crawler.js → Java /api/pipeline/tool → GetPackageContents (ISD)	Root packages: /HEC1/CP_CDD /HEC1/CP_NCDD	manifest.json: all sub-packages + objectQueue[]
Phase 2	cdd-crawler.js → Java /api/pipeline/tool → GetClass/Interface/etc (ISD)	objectQueue[] from manifest.json	knowledge/abap/ {type}/{name}.json (source + analysis)
Phase 3	cdd-crawler.js → Java /api/pipeline/tool → GetWhereUsed (ISD)	Interfaces + api-facade classes from phase 2	calledBy[] added to existing JSON records

Stage	Component	Input	Output
Graph	graph-builder.js (100% offline)	knowledge/abap/**/*.*.json	knowledge/graph/ index.json edges.json mind-map.json
Export	export-builder.js (100% offline)	knowledge/graph/ knowledge/abap/	cdd-system-prompt.md cdd-rag-chunks.jsonl
Sync	sync-java-cache.js → Java /api/pipeline/reload-cache	knowledge/abap/**/*.*.json	~/sap-ai-assistant/ cdd-cache/ (Java offline cache)
Runtime	Java MCP tools cdd_local_get_code cdd_local_search	AI tool calls (no SAP needed)	Source code + architecture from local cache

4. Java MCP Server

The Java server is the single binary that powers everything. It exposes both the MCP STUDIO protocol (for Claude Code) and a REST API on port 8080 (for the pipeline and web UI).

4.1 Starting the server

Build command

- cd jco-service && mvn clean package
- Output: jco-service/target/jco-service-1.0.0.jar
- Build time: ~2 min | Tests: 1,269 (no SAP required)

Start mode	Command	Purpose
MCP (for Claude Code)	java -jar target/jco-service-1.0.0.jar --mcp	STDIO protocol for AI assistants
Server (for pipeline)	java -jar target/jco-service-1.0.0.jar --server	REST API on port 8080
Script launcher	./start-sap-assistant.sh	Sets DYLD_LIBRARY_PATH automatically (macOS ARM64)

4.2 Key Java source files

File	Package	Purpose
PipelineController.java	.../web/	POST /api/pipeline/tool — invokes any MCP tool by name, reads output file, returns JSON. POST /api/pipeline/reload-cache — hot-reloads CDD cache index.
ChatController.java	.../web/	POST /api/chat (AI chat loop), GET /api/status, GET /api/download, GET /api/view

File	Package	Purpose
CddCodeCacheService.java	.../service/	Manages local offline cache. saveToCache(), search(), isCached(), reloadIndex(). Index stored as index.json.
CddCacheConfig.java	.../config/	Cache directory: \$CDD_CACHE_DIR or ~/sap-ai-assistant/cdd-cache
CddLocalGetCodeHandler.java	.../handlers/cdd/	MCP tool cdd_local_get_code. Cache-hit → return directly. Miss → fetch from SAP via JCo.
CddLocalSearchHandler.java	.../handlers/cdd/	MCP tool cdd_local_search. Text search on index (name, description, package).
CddKnowledgeBaseHandler.java	.../handlers/cdd/	MCP tool cdd_knowledge_base. Returns bundled cdd-knowledge-base.md or custom file.
McpToolRegistry.java	.../mcp/	Auto-discovers all @Component ToolHandler beans. getHandler(name) used by PipelineController.
AnthropicChatService.java	.../web/service/	AI chat loop. System prompt includes CDD/NCDD routing instructions: use local tools first.

4.3 REST API endpoints

Method	Path	Description
POST	/api/pipeline/tool	Invoke any MCP tool. Body: {tool, input}. Returns: {isError, text, filePath, fileContent}
POST	/api/pipeline/reload-cache	Hot-reload CDD cache index from disk. Returns: {success, message, entries}
POST	/api/chat	AI chat request (Claude). Body: {message}. Returns: {response}
GET	/api/status	Server health check. Returns 200 OK.
GET	/api/download?path=...	Download a file (restricted to /tmp and ~/Desktop)
GET	/api/view?path=...	View file contents (restricted to /tmp and ~/Desktop)
POST	/jco/http-proxy	Proxy authenticated HTTP requests to SAP via SNC session

4.4 CDD/NCDD offline tools

MCP Tool	When it uses SAP	Description
cdd_knowledge_base	Never	Returns architecture overview from bundled cdd-knowledge-base.md or ~/sap-ai-assistant/cdd-cache/knowledge-base.md
cdd_local_search	Never	Full-text search of cache index.json by name, description, package. Optionally filter by type.
cdd_local_get_code	Only on cache miss	Returns full source for a named object. Checks local cache first; falls back to SAP JCo if not cached.

4.5 Environment variables

Variable	Default	Purpose
CDD_CACHE_DIR	~/sap-ai-assistant/cdd-cache	Override the offline cache directory
DYLD_LIBRARY_PATH	(set by start-sap-assistant.sh)	JCo native library path (macOS ARM64)
SAP_ADT_DANGEROUS_OPERATIONS	false	Enable Create*/Delete* tools (default off for safety)

5. MCP Tools Reference (88 Tools)

All 88 tools are auto-registered via `McpToolRegistry` which collects all `@Component ToolHandler` beans. Tools follow the file-based output pattern: large results are written to `/tmp/sap-mcp/{systemId}_{client}/{category}/{file}` and the MCP response contains the file path plus a summary.

Category	Count	Tools
System	5	ListSystems, AddSystem, RemoveSystem, UpdateSystem, GetLandscapeSystems
Session	3	CreateSession, DestroySession, ListSessions
Read	29	GetClass, GetInterface, GetProgram, GetFunctionGroup, GetFunctionModule, GetInclude, GetCDSView, GetBehaviorDefinition, GetServiceDefinition, GetServiceBinding, GetClassInclude, GetVersionContent, GetVersionHistory, CompareVersions, GetWhereUsed, GetUsageSnippets, Search, GetPackageContents, GetTable, GetTableFields, GetStructure, GetStructureFields, GetDataElement, GetDomain, GetBopfBusinessObject, GetObjectDocumentation, GetObjectStatus, GetDiscovery, ListRuntimeErrors, GetRuntimeError
Write	24	SaveClass, SaveClassInclude, SaveClassTestInclude, SaveInterface, SaveProgram, SaveFunctionGroup, SaveFunctionModule, SaveInclude, SaveBehaviorDefinition, SaveServiceDefinition, LockObject, UnlockObject, ActivateObject, CheckSyntax, CreateClass, CreateInterface, CreateProgram, CreateFunctionGroup, CreateInclude, DeleteClass, DeleteInterface, DeleteProgram, DeleteFunctionGroup, DeleteInclude
Transport	3	ListTransportRequests, GetTransportRequests, GetTransportContents
Testing	9	RunAbapUnit, RunATC, GetATCFindings, ListATCResults, GetATCResult, ListATCCheckVariants, GetATCFindingDocumentation, GetCoverageResult, GetStatementCoverage
Data	3	PreviewTableData, PreviewCDSView, SelectSQLQuery
Debug	10	DebugStartSession, DebugSetBreakpoint, DebugDeleteBreakpoint, DebugWaitForBreakpoint, DebugGetStack, DebugGetVariables, DebugSetVariable, DebugStep, DebugStepToLine, DebugResume
BOPF	1	GetBopfBusinessObject
Format	1	FormatADTResponse
CDD/NCDD	3	cdd_knowledge_base, cdd_local_search, cdd_local_get_code (offline tools)

5.1 Handler pattern

Every tool extends `AbstractReadHandler` (read) or `AbstractWriteHandler` (write). Read handlers use `withTempSession()` to create and destroy a JCo session per call. Write handlers require an explicit session from `CreateSession`.

Tool invocation from Java (executeTool pattern):

```
Optional<ToolHandler> handler = toolRegistry.getHandler(toolName);
CallToolRequest request = new CallToolRequest(toolName, input);
McpSchema.CallToolResult result = handler.get().handle(null, request);
```

6. Knowledge Pipeline (Node.js)

The pipeline is a pure Node.js 18+ application with zero npm dependencies. It connects to the Java server (port 8080) during the crawl phase, then operates entirely offline.

6.1 Configuration — pipeline-config.json

```
{ "systemId": "ISD", "rootPackages": ["/HEC1/CP_CDD", "/HEC1/CP_NCDD"], "knowledgeDir":
"./knowledge", "javaServerUrl": "http://localhost:8080", "javaCacheDir":
"~/sap-ai-assistant/cdd-cache", "maxConcurrency": 5, "batchSize": 6 }
```

6.2 Running the pipeline

Command	What it does	SAP needed?
<code>node cdd-crawler.js --phase 1</code>	BFS over /HEC1/CP_CDD and /HEC1/CP_NCDD; builds objectQueue in manifest.json	YES
<code>node cdd-crawler.js --phase 2 --concurrency 5</code>	Downloads source for every object in queue; runs code-based analysis	YES
<code>node cdd-crawler.js --phase 2 --limit 1</code>	Crawl only 1 object — use for testing	YES
<code>node cdd-crawler.js --phase 3</code>	GetWhereUsed for all interfaces and api-facade/factory/singleton classes	YES
<code>node fix-manifest.js</code>	Sync manifest to disk (after interrupted crawl)	NO
<code>node fix-manifest.js --lock-phase1</code>	Set phase1Complete=true (skip phase 1 next run)	NO
<code>node graph-builder.js</code>	Build index.json, edges.json, mind-map.json, top-connected.json	NO
<code>node export-builder.js</code>	Build cdd-system-prompt.md and cdd-rag-chunks.jsonl	NO
<code>node sync-java-cache.js</code>	Copy source files to Java cache, reload index via HTTP	NO
<code>node sync-java-cache.js --dry-run</code>	Preview what would be written without writing	NO

Full first-time setup sequence:

```
# 1. Start Java server (SNC auth to SAP ISD required)
cd jco-service && java -jar target/jco-service-1.0.0.jar --server

# 2. Run the pipeline (in another terminal)
cd pipeline
node cdd-crawler.js --phase 1 # ~30 min
node cdd-crawler.js --phase 2 --concurrency 5 # ~2-4 hrs
node cdd-crawler.js --phase 3 # ~30 min
```

```
node graph-builder.js
node export-builder.js
node sync-java-cache.js

# 3. Everything is now offline – stop the Java server
# Future sessions need only: java -jar ... --mcp
```

6.3 Resuming after interruption

```
node fix-manifest.js # rebuild crawled map from disk
node cdd-crawler.js --phase 2 --concurrency 5 # resume
```

6.4 Object JSON record format

```
{ "name": "/HEC1/CL_CDD_EMAIL", "type": "class", "package": "/HEC1/CP_CDD_APP", "source": "<full ABAP source>", "lines": 1407, "sizeBytes": 61000, "analysis": { "role": "Email notification orchestrator for CDD state changes", "patterns": ["notification-handler", "bcs-sender"], "keyMethods": ["EVENT_CDD", "SEND_MAIL"], "dependencies": ["/HEC1/IF_CDD_EMAIL", "/HEC1/CL_PROV_UTILITY"], "tables": ["/HEC1/CDD_CUSTOM", "/HEC1/CDD_ACTION"], "summary": "...", "calledBy": ["..."], // added by phase 3 "crawledAt": "2026-04-17T..." }
```

6.5 Code analysis patterns detected

Pattern	Detection rule
singleton	Source contains get_instance(
factory	Name contains _FACTORY or source contains create_*(calls
api-facade	Name contains _MANAGER
notification-handler	Source references cl_bcs, lif_sender, or cl_document_bcs
validator	Name contains _VALID and source references if_message_handler or check_*
status-machine	Name contains status/state/workflow AND source has set_status/change_status
rap-behavior	Source has IF_X-METHOD pattern AND 'implementation' keyword
rap-event-handler	Source has ON_*EVENT pattern
automation-calculator	Name contains _CALC or source has calculate(calls
runnable	Source references IF_OO_ADT_CLASSRUN

6.6 Pipeline files

File	Description
cdd-crawler.js	Main crawler. Phases 1/2/3. Calls Java /api/pipeline/tool for SAP data. Regex-based analyzeSource() for offline analysis.
fix-manifest.js	Scans knowledge/abap/ on disk, rebuilds crawled map, removes done items from objectQueue. Flags: --lock-phase1, --unlock-phase1
graph-builder.js	Reads all JSON records. Writes index.json (node registry), edges.json (depends-on + calls), mind-map.json (package tree), top-connected.json (top 20 by degree).
export-builder.js	Reads graph + records. Writes cdd-system-prompt.md (rich architecture doc, 2000+ lines) and cdd-rag-chunks.jsonl (one chunk per object for vector search).

File	Description
sync-java-cache.js	Reads knowledge/abap/**/*.json. Copies source to ~/sap-ai-assistant/cdd-cache/{type}/{name}.abap. Updates index.json. Calls /api/pipeline/reload-cache to hot-reload.
pipeline-config.json	Centralised config: systemId, rootPackages, knowledgeDir, javaServerUrl, javaCacheDir, maxConcurrency, batchSize.
package.json	npm scripts: phase1, phase2, phase3, fix, graph, export. No external dependencies. Node 18+ required.

7. CDD/NCDD Domain Knowledge

CDD (Configuration Delivery Document) is a SAP Ariba/IBP integration object type. There are two generations: **CDD v1** (BOPF-based) in /HEC1/CP_CDD and **CDD 2.0 / NCDD** (RAP-based) in /HEC1/CP_NCDD.

7.1 CDD v1 vs CDD 2.0 comparison

Aspect	CDD v1 (/HEC1/CP_CDD)	CDD 2.0 NCDD (/HEC1/CP_NCDD)
Framework	BOPF (Business Object Processing Framework)	RAP (RESTful ABAP Programming)
UI	Web Dynpro ABAP	Fiori Elements (OData V4)
Entry point	CL_CDD_MANAGER.get_instance()	BPCL_I_NCDD_WORKLIST
Automation	Manual / none	Factory + calculators pattern
Draft support	No	Yes (RAP managed scenario)
IBP integration	Direct RFC/BAPI calls	Via RAP action/determinations
Notification	CL_CDD_EMAIL.EVENT_CDD()	NCDD notification CDS views

7.2 Key classes — CDD v1

Class / Object	Package	Role
/HEC1/CL_CDD_MANAGER	/HEC1/CP_CDD_APP	Singleton. Primary entry point for CDD v1 operations.
/HEC1/CL_CDD_EMAIL	/HEC1/CP_CDD_APP	Notification-handler. Orchestrates all email notifications via CL_BCS.
/HEC1/CL_CDD_CUSTOMIZATION	/HEC1/CP_CDD_CUSTOMIZATION	Reads /HEC1/CDD_CUSTOM table. Feature flags and email recipients.
/HEC1/CL_PROV_UTILITY	/HEC1/CP_CDD	is_feature_enabled(iv_feature_id). Central feature toggle.
/HEC1/IF_CDD_EMAIL	/HEC1/CP_CDD_APP	Interface for email notification contract.
/HEC1/IF_CDD_MANAGER	/HEC1/CP_CDD	Interface for CDD manager contract.

7.3 Key classes — CDD 2.0 (NCDD)

Class / Object	Package	Role
/HEC1/CL_NCDD_AUTO_FACTORY	/HEC1/CP_NCDD_AUTOMATIO N	Factory. Dispatches to correct automation calculator by level_id.
/HEC1/IF_NCDD_AUTO_CALC	/HEC1/CP_NCDD_AUTOMATIO N	Interface. Contract for all automation calculators.
/HEC1/I_NCDD_WORKLIST	/HEC1/CP_NCDD	RAP interface BDEF for NCDD Worklist.
/HEC1/C_NCDD_WORKLIST	/HEC1/CP_NCDD	RAP consumption BDEF for NCDD Worklist.
/HEC1/BPCL_I_NCDD_WORKLIST	/HEC1/CP_NCDD_APP	RAP behavior class. Implements BDEF methods.

7.4 Notification system (CDD v1)

Flow: EVENT_CDD(event_id) → is_feature_enabled() → recipients from /HEC1/CDD_CUSTOM → CL_BCS send → log /HEC1/CDD_ACTION

Event ID	Trigger
CDD_NOTIF_26285	CDD requested
CDD_NOTIF_33189	Sent for approval
CDD_NOTIF_26257	DR requested
(others)	rejected_int_gdo, rejected_int, approved, maintain_subcontractor

7.5 Customization tables

Table	Purpose
/HEC1/CDD_CUSTOM	Generic params + email recipients (key1/key2/key3 structure)
/HEC1/CDDBOPFCNF	BOPF field mapping configuration
/HEC1/CDD_CR	CR customization: lead times, durations
/HEC1/CDD_ACTION	Audit log: all state changes and notification events
/HEC1/NCDD_AUTO_S_CALC_CUS T	NCDD automation calculator configuration (level_id → calc class)

7.6 Feature toggle mechanism

CDD v1 uses **no dedicated toggle table**. All feature gating is via /HEC1/CL_PROV_UTILITY=>is_feature_enabled(iv_feature_id). Feature IDs are string constants passed at the call site.

7.7 Automation calculator pattern (NCDD)

```
/HEC1/CL_NCDD_AUTO_FACTORY.create_calculator(level_id)
→ returns /HEC1/IF_NCDD_AUTO_CALC instance
→ caller invokes IF_NCDD_AUTO_CALC~calculate()
→ result stored via RAP action/determination
```

7.8 /cdd-knowledge slash command

File: `.claude/commands/cdd-knowledge.md`. When invoked as `/cdd-knowledge`, injects the full `knowledge/exports/cdd-system-prompt.md` into the AI context (2000+ lines of architecture knowledge).

```
# In .claude/commands/cdd-knowledge.md:
$(cat ~/Desktop/sap-ai-assistant/knowledge/exports/cdd-system-prompt.md 2>/dev/null || echo 'Run
export-builder.js first.')
```

8. SAP System Configuration

SAP systems are configured in `.sap-systems.json` at the project root. This file is gitignored and contains SNC credentials. **Only SNC authentication is supported**; basic auth (username/password) was removed in v2.0.0.

8.1 .sap-systems.json format

```
{ "systems": [ { "id": "ISD", "url": "https://isd.example.com:44300", "client": "100", "sysnr": "00",
"snc_partnername": "p/secude:CN=ISD, O=SAP-AG, C=DE", "snc_qop": 9, "description": "ISD Development
System" } ], "default": "ISD" }
```

8.2 JCo SNC setup

- Native JCo library must be in DYLD_LIBRARY_PATH (macOS ARM64) or LD_LIBRARY_PATH (Linux).
- `sapjco3.jar` must precede the application JAR in the classpath.
- SNC uses Kerberos/GSS-API — requires valid Kerberos ticket (kinit) before connecting.
- Node.js pipeline cannot do SNC directly — it always calls the Java server as proxy.
- The MCP tool `AddSystem` can register new systems; `GetLandscapeSystems` reads SAP GUI landscape file.

8.3 ABAP type to ADT URL mapping

ABAP Type	ADT Endpoint
class	/sap/bc/adt/oo/classes/{name}/source/main
interface	/sap/bc/adt/oo/interfaces/{name}/source/main
program	/sap/bc/adt/programs/programs/{name}/source/main
function_group	/sap/bc/adt/functions/groups/{name}/source/main
cds_view	/sap/bc/adt/ddic/ddl/sources/{name}/source/main
behavior_definition	/sap/bc/adt/bo/behaviordefinitions/{name}/source/main

9. Offline Mode Reference

After completing the initial crawl, the system runs **100% without internet, without Claude proxy, and without SAP connectivity**. This section explains exactly what is and is not required.

9.1 What is needed offline

Component	Required offline?	Notes
Java JAR (<code>jco-service-1.0.0.jar</code>)	YES — run as <code>--mcp</code>	Serves all 88 tools including offline CDD tools
<code>~/sap-ai-assistant/cdd-cache/</code>	YES	<code>index.json</code> + source files. Created by <code>sync-java-cache.js</code>

Component	Required offline?	Notes
knowledge/exports/cdd-system-prompt.md	YES (for /cdd-knowledge)	Created by export-builder.js
SAP ISD system	NO	Only needed during initial crawl phases 1-3
Internet / Claude proxy	NO	No external calls at runtime
Node.js pipeline	NO	Only needed during crawl and graph rebuild
Kerberos ticket (kinit)	NO	Only needed when making SAP connections

9.2 Offline cache directory structure

```
~/sap-ai-assistant/cdd-cache/
■■■■ index.json ← object registry (name, type, pkg, path, cachedAt, sizeBytes)
■■■■ knowledge-base.md ← optional custom knowledge base file
■■■■ class/
■ ■■■ _HEC1_CL_CDD_EMAIL.abap
■ ■■■ ...
■■■■ interface/
■ ■■■ ...
■■■■ cds_view/
■ ■■■ *.ddl
■■■■ behavior_definition/
■■■■ *.abap
```

9.3 AI system prompt routing (offline tools first)

The AnthropicChatService system prompt instructs the AI to always try local tools first:

For CDD/NCDD questions – use in this order:

1. cdd_knowledge_base (architecture overview, no SAP needed)
2. cdd_local_search (find objects by keyword, no SAP needed)
3. cdd_local_get_code (get source; cache hit = no SAP needed)
4. GetClass / GetInterface etc. (only if object NOT in cache)

10. Troubleshooting

10.1 Common issues

Symptom	Likely cause	Fix
Pipeline: 'server not reachable at :8080'	Java server not running	cd jco-service && java -jar target/jco-service-1.0.0.jar --server
Phase 1 produces empty objectQueue	Package XML parsing missed objects	Check raw fileContent returned by GetPackageContents; adjust parsePackageContents() regex
Crawl interrupted mid-phase-2	Process killed, network error	node fix-manifest.js && node cdd-crawler.js --phase 2 --concurrency 5
cdd_local_get_code returns empty	Object not in cache and SAP unreachable	Run sync-java-cache.js after crawl to populate cache
POST /api/pipeline/reload-cache returns 404	Old JAR (before PipelineController was added)	mvn clean package in jco-service, restart server

Symptom	Likely cause	Fix
'javax.annotation does not exist' build error	Wrong Jakarta import (Java 17 / Spring Boot 3)	Use jakarta.annotation.PostConstruct (not javax.*)
JCo 'SNC name could not be mapped' error	SNC partner name mismatch or missing Kerberos ticket	Run kinit; verify snc_partnername in .sap-systems.json
/cdd-knowledge returns 'Run export-builder.js first'	knowledge/exports/ not generated	node graph-builder.js && node export-builder.js
sync-java-cache.js reports 0 added	knowledge/abap/ is empty (crawl not done)	Complete phase 2 first, then sync

10.2 Verification checklist

- Java server: `curl http://localhost:8080/api/status → 200 OK`
- Pipeline tool endpoint: `curl -X POST http://localhost:8080/api/pipeline/tool -H 'Content-Type: application/json' -d '{"tool": "ListSystems", "input": {}}' | jq .text`
- Phase 1 done: `cat knowledge/crawl/manifest.json | jq '{phase1Complete, queueLen: (.objectQueue | length)}'`
- Phase 2 done: `ls knowledge/abap/classes/ | wc -l` (expect 100+)
- Graph done: `ls knowledge/graph/` (index.json, edges.json, mind-map.json, top-connected.json)
- Export done: `wc -l knowledge/exports/cdd-system-prompt.md` (expect 2000+ lines)
- Java cache: `ls ~/sap-ai-assistant/cdd-cache/class/ | wc -l` (expect same as knowledge/abap/classes/)
- Offline search: (MCP) `cdd_local_search query='CDD_EMAIL' → returns /HEC1/CL_CDD_EMAIL`
- Offline code: (MCP) `cdd_local_get_code name='/HEC1/CL_CDD_EMAIL' type='class' → ABAP source`
- /cdd-knowledge: invoke in Claude Code → returns architecture overview without SAP call

11. Quick Reference

11.1 Key file paths

What	Path
Java JAR	jco-service/target/jco-service-1.0.0.jar
Start script	start-sap-assistant.sh
SAP systems config	.sap-systems.json (gitignored)
Pipeline config	pipeline/pipeline-config.json
Crawl progress	knowledge/crawl/manifest.json
Crawled ABAP objects	knowledge/abap/{class,interface,program,...}/
Dependency graph	knowledge/graph/
System prompt for AI	knowledge/exports/cdd-system-prompt.md
Java offline cache	~/sap-ai-assistant/cdd-cache/
Cache index	~/sap-ai-assistant/cdd-cache/index.json
Slash command	.claude/commands/cdd-knowledge.md
Bundled knowledge base	jco-service/src/main/resources/cdd-knowledge-base.md
This document	docs/SAP-AI-Assistant-Complete-Reference.pdf

11.2 Most important commands

```
# Build Java server
cd jco-service && mvn clean package

# Run as MCP server (for Claude Code)
java -jar jco-service/target/jco-service-1.0.0.jar --mcp

# Run as REST server (for pipeline)
java -jar jco-service/target/jco-service-1.0.0.jar --server

# Full crawl (run once, SAP + Java server required)
cd pipeline
node cdd-crawler.js --phase 1
node cdd-crawler.js --phase 2 --concurrency 5
node cdd-crawler.js --phase 3
node graph-builder.js && node export-builder.js && node sync-java-cache.js

# Resume interrupted crawl
node fix-manifest.js && node cdd-crawler.js --phase 2 --concurrency 5

# Check offline tools (no SAP needed)
# (via Claude Code MCP) cdd_local_search query='email'
# (via Claude Code MCP) cdd_local_get_code name='/HEC1/CL_CDD_EMAIL' type='class'
```

End of Document

This document covers the complete SAP AI Assistant project as of 2026-04-17. For the latest source of truth, read the source files directly. Key starting points: `CLAUDE.md` (project guidance), `pipeline/pipeline-config.json` (configuration), and `jco-service/src/main/java/com/sapjco/mcp/web/PipelineController.java` (pipeline API).